SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

NAME:RAGUL.M

REG NO:192524066

SUBJECT:PYTHON PROGRAMMING

CODE:CSA0802

CO:2 AT:1

GITHUB LINK: https://github.com/192524066simats-netizen/CSA0802/tree/main/ASSISGNMENT%20TOOL/CO1

TASK 1 – Decisions with Many Boundaries

Aim:

To develop a Python program using separate if statements and if-elif-else statements to determine electricity subsidy eligibility based on electricity consumption, ration card availability and age.

Algorithm:

1. Start the program.
2. Get electricity units, ration card status and age.
3. Check whether consumption is 100 units or less.
4. Check whether consumption is between 101 and 200 units with a ration card.
5. Check whether consumption is above 200 units and age is 80 or above.
6. Display the appropriate subsidy result.
7. Repeat the process for different households.
8. Stop the program.

Pseudocode:

```
START
FOR each household
    INPUT units, ration, age
    SET eligible = FALSE
   IF units <= 100
       PRINT Full Subsidy
```

SET eligible = TRUE

END IF

IF units > 100 AND units <= 200 AND ration

PRINT Partial Subsidy

SET eligible = TRUE

END IF

IF units > 200 AND age >= 80

PRINT Senior Citizen Exception

SET eligible = TRUE

END IF

IF eligible == FALSE

PRINT No Subsidy

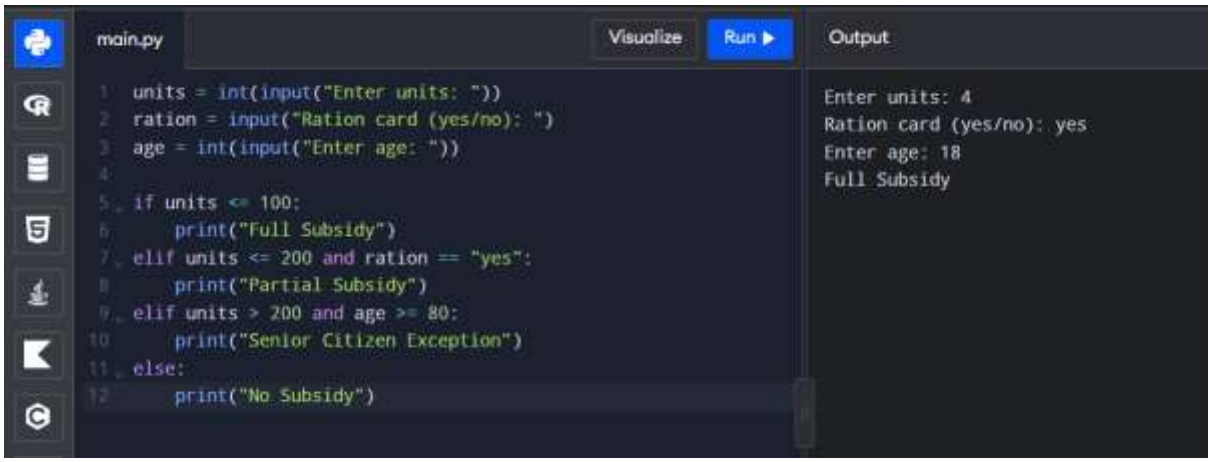
END IF

END FOR

STOP

Result:

Thus, the Python program was successfully executed using separate if statements and if-elif-else statements to determine electricity subsidy eligibility.



TASK 2 – Loops that Count Attempts

Aim:

To implement OTP verification using while and for loops and demonstrate break, continue, pass and loop else statements.

Algorithm:

1. Start the program.

2. Initialize the correct OTP and attempt counter.
3. Accept the OTP from the user.
4. Check whether the entered OTP is blank.
5. If blank, skip the attempt counter update.
6. If the OTP is correct, display successful verification.
7. If incorrect, increase the attempt counter.
8. Lock the account after three incorrect attempts.
9. Stop the program.

Pseudocode:

```
START
SET correct_otp = "1234"
SET attempts = 0

WHILE attempts < 3
   INPUT entered_otp

   IF entered_otp is blank
      PRINT Blank Entry
      CONTINUE
   END IF

   INCREASE attempts

   IF entered_otp == correct_otp
      PRINT OTP Verified
      BREAK
   ELSE
      PRINT Incorrect OTP
   END IF
```

ELSE

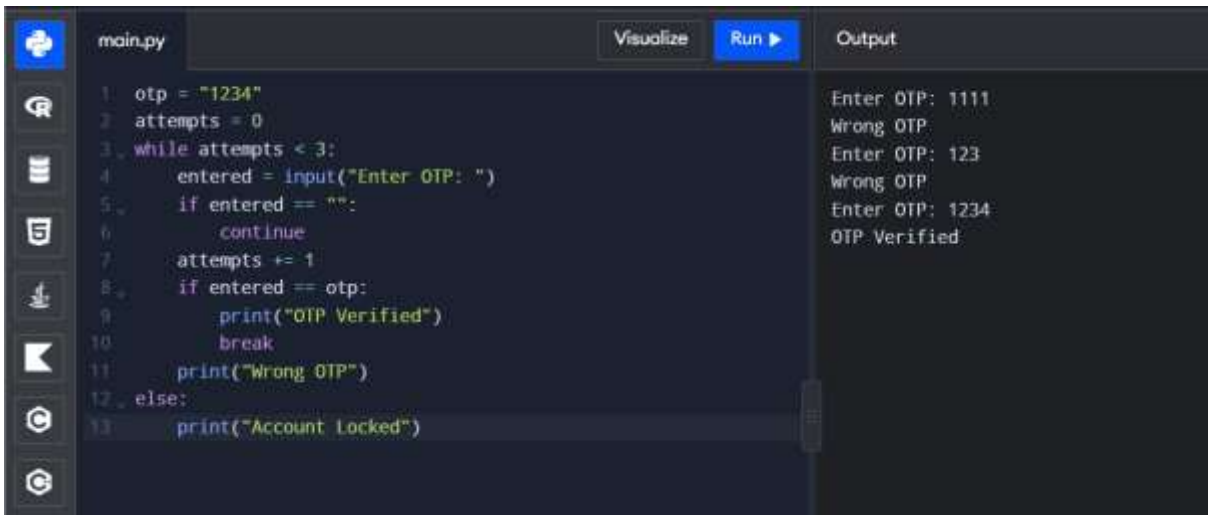
PRINT Account Locked

END WHILE

STOP

Result:

Thus, the OTP verification program was successfully executed using while and for loops with loop control statements to manage verification attempts and account locking.



TASK 3 – Vehicle Registration Number Validator

Aim:

To develop a Python program to normalize, validate and format vehicle registration numbers using string operations and slicing without regular expressions.

Algorithm:

1. Start the program.
2. Get the vehicle registration number.
3. Remove unwanted spaces and hyphens.
4. Convert the registration number into uppercase.
5. Check the total length.
6. Validate the state code using isalpha().
7. Validate the district number using isdigit().
8. Validate the series and registration number.
9. Display whether the registration number is valid or invalid.

10. Format valid registration numbers.

11. Stop the program.

Pseudocode:

```
START

FUNCTION normalise(text)
    REMOVE spaces and hyphens
    CONVERT text to uppercase
    RETURN text
END FUNCTION

FUNCTION is_valid(reg)
    IF length is not 9 or 10
        RETURN FALSE
    END IF

    IF first two characters are not alphabets
        RETURN FALSE
    END IF

    IF next two characters are not digits
        RETURN FALSE
    END IF

    IF series is not alphabetic
        RETURN FALSE
    END IF

    IF last four characters are not digits
```

RETURN FALSE

END IF

RETURN TRUE

END FUNCTION

INPUT registration number

NORMALISE registration number

IF is_valid(registration number)

PRINT VALID

FORMAT registration number

ELSE

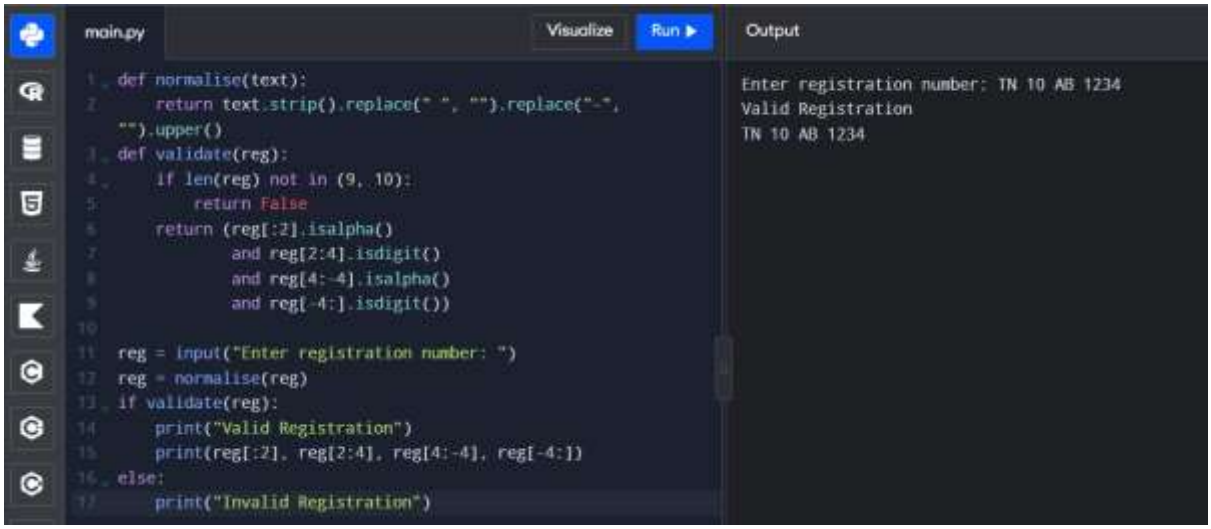
PRINT INVALID

END IF

STOP

Result:

Thus, the vehicle registration number validator was successfully implemented using Python string methods and slicing to normalize, validate and format registration numbers.



TASK 4 – Building Long Text: + or join()?

Aim:

To compare different string-building methods using repeated concatenation, list with join() and f-strings, and measure their execution time.

Algorithm:

1. Start the program.
2. Import the time module.
3. Generate a list of customer statement items.
4. Build the statement using repeated +.
5. Build the same statement using list and join().
6. Build the statement using f-strings and join().
7. Measure the execution time of each method.
8. Compare the generated outputs.
9. Display execution times and results.
10. Stop the program.

Pseudocode:

```
START

IMPORT time module
CREATE list of statement items

FUNCTION build_plus(items)
   SET text = ""
   FOR each item
      ADD item to text using +
   END FOR
   RETURN text
END FUNCTION

FUNCTION build_join(items)
```

```
    CREATE empty list
    FOR each item
        APPEND item to list
    END FOR
    JOIN all items
    RETURN result
END FUNCTION

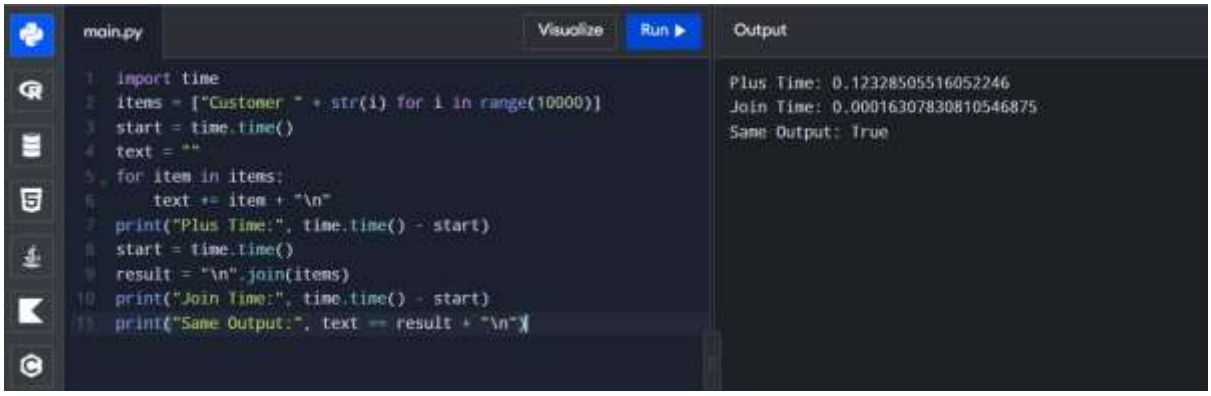
FUNCTION build_fstring(items)
    FORMAT each item using f-string
    JOIN all formatted items
    RETURN result
END FUNCTION

FOR each input size
    MEASURE execution time of all methods
    COMPARE outputs
    DISPLAY execution times
END FOR

STOP
```

Result:

Thus, the three string-building methods were successfully implemented and compared. The join() method provides an efficient approach for combining large numbers of strings.



TASK 5 – Functions, Parameters and Scope

Aim:

To demonstrate Python functions, positional parameters, keyword parameters, default parameters, return values and variable scope, and to identify and correct the mutable default argument problem.

Algorithm:

1. Start the program.
2. Define a function with a mutable default list.
3. Add items to the order list.
4. Call the function multiple times.
5. Observe how previous customer orders are retained.
6. Define a corrected function using None as the default value.
7. Create a new list whenever the argument is None.
8. Add items independently for each customer.
9. Demonstrate different parameter types and variable scopes.
10. Display the results.
11. Stop the program.

Pseudocode:

START

FUNCTION add_item_buggy(item, order = [])

APPEND item to order

RETURN order

END FUNCTION

```
CALL add_item_buggy("Tea")
CALL add_item_buggy("Vada")

FUNCTION add_item(item, order = None)
   IF order is None
      CREATE new empty list
   END IF

   APPEND item to order
   RETURN order
END FUNCTION

CALL add_item("Tea")
CALL add_item("Vada")

DISPLAY all results

STOP
```

Result:

Thus, the mutable default parameter problem was identified and corrected using None. The program successfully demonstrated independent customer orders, parameter types, return

values and variable scope.



TASK 6 – Recursion and Its Limits

Aim:

To calculate the number of ways to climb temple steps using normal recursion, memoisation and iteration, and compare their performance and memory requirements.

Algorithm:

1. Start the program.
2. Define the base cases for zero and one step.
3. Implement the recursive function.
4. Calculate the result using normal recursion.
5. Implement memoisation using lru_cache.
6. Store previously calculated results.
7. Implement an iterative solution using a loop.
8. Calculate results for steps 1 to 6.
9. Compare function calls for normal recursion and memoisation.
10. Test the recursion limit for 1000 steps.
11. Display the results.
12. Stop the program.

Pseudocode:

START

```
FUNCTION ways_naive(n)
    IF n <= 1
        RETURN 1
    END IF

    RETURN ways_naive(n-1) + ways_naive(n-2)
END FUNCTION

FUNCTION ways_memo(n)
    IF result already exists
        RETURN stored result
    END IF

    IF n <= 1
        RETURN 1
    END IF

    CALCULATE ways(n-1) + ways(n-2)
    STORE result
    RETURN result
END FUNCTION

FUNCTION ways_loop(n)
    SET a = 1
    SET b = 1

    FOR i from 2 to n
        SET a = b
        SET b = a + previous value of b
```

END FOR

RETURN b

END FUNCTION

TEST all three methods

COMPARE results and function calls

STOP

Result:

Thus, the number of ways to climb temple steps was successfully calculated using recursion, memoisation and iteration. Memoisation reduced repeated calculations, while the iterative approach avoided recursion depth limitations.

```python
from functools import lru_cache
def ways(n):
    if n <= 1:
        return 1
    return ways(n-1) + ways(n-2)
def memo(n):
    if n <= 1:
        return 1
    return memo(n-1) + memo(n-2)
def iterative(n):
    a, b = 1, 1
    for i in range(2, n+1):
        a, b = b, a+b
    return b
n = int(input("Enter steps: "))
print("Normal Recursion:", ways(n))
print("Memoisation:", memo(n))
print("Iteration:", iterative(n))
```

```
Enter steps: 5
Normal Recursion: 8
Memoisation: 8
Iteration: 8
```

TASK 7 – Diagnose, Redesign and Validate

Aim:

To identify the performance bottleneck in a Python program, redesign the algorithm using an efficient string-building technique and validate the improvement through execution-time measurements.

Algorithm:

1. Start the program.
2. Identify the original string concatenation method.

3. Generate customer statement data.
4. Execute the original program using repeated +.
5. Measure its execution time.
6. Redesign the program using join().
7. Execute the redesigned program.
8. Measure its execution time.
9. Compare both outputs.
10. Calculate the performance improvement.
11. Analyse time and space complexity.
12. Display the final comparison.
13. Stop the program.

Pseudocode:

```
START

CREATE customer statement list

FUNCTION original(items)
   SET text = ""

   FOR each item
      text = text + item
   END FOR

   RETURN text
END FUNCTION

FUNCTION redesigned(items)
   JOIN all items into one string
   RETURN result
```

END FUNCTION

MEASURE original execution time

MEASURE redesigned execution time

COMPARE both outputs

IF outputs are equal

    PRINT Validation Successful

ELSE

    PRINT Validation Failed

END IF

CALCULATE performance improvement

DISPLAY execution times

DISPLAY complexity comparison

STOP

Result:

Thus, the original string concatenation program was successfully redesigned using the join() method. Both programs produced identical outputs, and their execution times were measured

to evaluate the performance improvement.

